Francisco Acuña - 20220565

Report

Pasos para la Creación de Procesos

La creación de procesos en los sistemas operativos, particularmente en sistemas tipo Unix, sigue estos pasos clave:

Fork System Call:

- The parent process creates a child process by calling fork()
- This creates an almost identical copy of the parent process
- The child receives a copy of the parent's address space, file descriptors, and resources
- The only difference is that fork() returns the child's PID to the parent and 0 to the child

Exec System Call:

- After forking, the child often calls one of the exec family functions
- This replaces the child's memory space with a new program
- The process keeps the same PID but runs different code

Wait System Call:

- The parent process can call wait() or waitpid() to wait for child completion
- This allows synchronization between parent and child processes
- The parent receives the child's exit status when it terminates

Sincronización de Procesos

Synchronization mechanisms ensure proper coordination between concurrent processes:

Mutexes (Mutual Exclusion):

- Protect critical sections by allowing only one process to access a resource
- Basic operations: lock (acquire) and unlock (release)
- If a process tries to lock an already locked mutex, it blocks until the mutex is released

Semaphores:

- More general synchronization primitives with a counter value
- Binary semaphores (values 0/1) function similar to mutexes
- Counting semaphores control access to finite resources

- Operations: wait (decrement) and signal (increment)

Condition Variables:

- Allow processes to wait for a specific condition to become true
- Used with mutexes to handle complex synchronization scenarios
- Operations: wait, signal, and broadcast

Barriers:

- Force a set of processes to wait until all reach a certain point
- Useful for parallel algorithms with distinct phases

Mecanismos de IPC (Comunicación entre Procesos)

IPC allows processes to exchange data and synchronize actions:

Pipes:

- Unidirectional communication channels
- Usually used between parent and child processes
- Anonymous pipes exist only while the creating process lives
- Named pipes (FIFOs) exist as special files in the filesystem

Message Queues:

- Allow processes to exchange discrete messages
- Messages can have types and priorities
- Processes can send and receive messages without being synchronized

Shared Memory:

- Fastest IPC method where processes share a region of memory
- Requires explicit synchronization (mutexes or semaphores)
- Created via system calls like shmget() and shmat() (POSIX)

Memory-mapped Files:

- Map file contents directly into a process's address space
- Can be used for IPC when multiple processes map the same file

Sockets:

- Communication endpoints for network or local communication
- Allow communication between processes on different machines

- Unix domain sockets enable efficient local IPC

Screenshot: